Training Deep Neural Networks (Tiếp)

Trong buổi này, chúng ta tiếp tục làm quen với một số kỹ thuật huấn luyện mạng nơ-ron

In [4]:
```
import torch
import torch.nn as nn
import torch.optim as optim
import numpy as np
import glob
import cv2
import torch.nn.functional as F
from torch.autograd import Variable

import torchvision
import torchvision.transforms as transforms

from torch.nn import CrossEntropyLoss, Dropout, Softmax, Linear, Conv2d, LayerNorm
import matplotlib.pyplot as plt
```

1. Cài đặt BatchNorm

In [ ]:
```
def compare_bn(bn1, bn2):
    err = False
    if not torch.allclose(bn1.running_mean, bn2.running_mean):
        print('Diff in running_mean: {} vs {}'.format(
            bn1.running_mean, bn2.running_mean))
        err = True

    if not torch.allclose(bn1.running_var, bn2.running_var):
        print('Diff in running_var: {} vs {}'.format(
            bn1.running_var, bn2.running_var))
        err = True

    if bn1.affine and bn2.affine:
        if not torch.allclose(bn1.weight, bn2.weight):
            print('Diff in weight: {} vs {}'.format(
                bn1.weight, bn2.weight))
            err = True

        if not torch.allclose(bn1.bias, bn2.bias):
            print('Diff in bias: {} vs {}'.format(
                bn1.bias, bn2.bias))
            err = True

    if not err:
        print('All parameters are equal!')


class MyBatchNorm2d(nn.BatchNorm2d):
    def __init__(self, num_features, eps=1e-5, momentum=0.1,
                 affine=True, track_running_stats=True):
```

```
        super(MyBatchNorm2d, self).__init__(
            num_features, eps, momentum, affine, track_running_stats)

    def forward(self, input):
        self._check_input_dim(input)

        exponential_average_factor = 0.0

        if self.training and self.track_running_stats:
            if self.num_batches_tracked is not None:
                self.num_batches_tracked += 1
                if self.momentum is None:  # use cumulative moving average
                    exponential_average_factor = 1.0 / float(self.num_batches_track
                else:  # use exponential moving average
                    exponential_average_factor = self.momentum

        # calculate running estimates
        if self.training:
            mean = input.mean([0, 2, 3])
            # use biased var in train
            var = input.var([0, 2, 3], unbiased=False)
            n = input.numel() / input.size(1)
            with torch.no_grad():
                self.running_mean = exponential_average_factor * mean\
                    + (1 - exponential_average_factor) * self.running_mean
                # update running_var with unbiased var
                self.running_var = exponential_average_factor * var * n / (n - 1)\
                    + (1 - exponential_average_factor) * self.running_var
        else:
            mean = self.running_mean
            var = self.running_var

        input = (input - mean[None, :, None, None]) / (torch.sqrt(var[None, :, None
        if self.affine:
            input = input * self.weight[None, :, None, None] + self.bias[None, :, N

        return input


# Init BatchNorm layers
my_bn = MyBatchNorm2d(3, affine=True)
bn = nn.BatchNorm2d(3, affine=True)

compare_bn(my_bn, bn)  # weight and bias should be different
# Load weight and bias
my_bn.load_state_dict(bn.state_dict())
compare_bn(my_bn, bn)

# Run train
for _ in range(10):
    scale = torch.randint(1, 10, (1,)).float()
    bias = torch.randint(-10, 10, (1,)).float()
    x = torch.randn(10, 3, 100, 100) * scale + bias
    out1 = my_bn(x)
    out2 = bn(x)
    compare_bn(my_bn, bn)
```

```
    torch.allclose(out1, out2)
    print('Max diff: ', (out1 - out2).abs().max())

# Run eval
my_bn.eval()
bn.eval()
for _ in range(10):
    scale = torch.randint(1, 10, (1,)).float()
    bias = torch.randint(-10, 10, (1,)).float()
    x = torch.randn(10, 3, 100, 100) * scale + bias
    out1 = my_bn(x)
    out2 = bn(x)
    compare_bn(my_bn, bn)

    torch.allclose(out1, out2)
```

2. Cài đặt chiến lược thay đổi tốc độ học: Warm-up + Cosine Annealing LR

In [6]:

```
def load_data(data_dir="./data"):
    transform = transforms.Compose([
        transforms.ToTensor(),
        transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5)),
    ])

    trainset = torchvision.datasets.CIFAR10(
        root=data_dir, train=True, download=True, transform=transform)

    testset = torchvision.datasets.CIFAR10(
        root=data_dir, train=False, download=True, transform=transform)

    return trainset, testset

class Net(nn.Module):
    def __init__(self, l1=120, l2=84):
        super(Net, self).__init__()
        self.conv1 = nn.Conv2d(3, 6, 5)
        self.pool = nn.MaxPool2d(2, 2)
        self.conv2 = nn.Conv2d(6, 16, 5)
        self.fc1 = nn.Linear(16 * 5 * 5, l1)
        self.fc2 = nn.Linear(l1, l2)
        self.fc3 = nn.Linear(l2, 10)
        self.softmax = nn.Softmax()

    def forward(self, x):
        x = self.pool(F.relu(self.conv1(x)))
        x = self.pool(F.relu(self.conv2(x)))
        x = x.view(-1, 16 * 5 * 5)
        x = F.relu(self.fc1(x))
        x = F.relu(self.fc2(x))
        x = self.fc3(x)
        x = self.softmax(x)
```

```
        return x
```

In [ ]:
```
trainset, testset = load_data('./data')
trainloader = torch.utils.data.DataLoader(
    trainset,
    batch_size=128,
    shuffle=True,
)

epochs = 50
warm_epoch = 5
init_lr = 1e-1
last_lr = 1e-5
T_max = epochs
T_cur = 0
lr_list = [0]

net = Net()
device = "cpu"
if torch.cuda.is_available():
    device = "cuda:0"
    if torch.cuda.device_count() > 1:
        net = nn.DataParallel(net)
net.to(device)

criterion = nn.CrossEntropyLoss()
optimizer = optim.SGD(net.parameters(), lr=init_lr, momentum=0.9)
```

In [ ]:
```
for epoch in range(1, epochs+1):  # loop over the dataset multiple times
    running_loss = 0.0
    epoch_steps = 0
    T_cur += 1

    # warm-up
    if epoch <= warm_epoch:
        optimizer.param_groups[0]['lr'] = (1.0 * epoch) / warm_epoch  * init_lr
    else:
        # cosine annealing lr
        optimizer.param_groups[0]['lr'] = last_lr + (init_lr - last_lr) * (1 + np.c

    for i, data in enumerate(trainloader, 0):
        # get the inputs; data is a list of [inputs, labels]
        inputs, labels = data
        inputs, labels = inputs.to(device), labels.to(device)

        # zero the parameter gradients
        optimizer.zero_grad()

        # forward + backward + optimize
        outputs = net(inputs)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()
```

```
        # print statistics
        running_loss += loss.item()
        epoch_steps += 1
        if i + 1 == len(trainloader):
            print("[Epoch %d] loss: %.3f" % (epoch, running_loss / epoch_steps))
            running_loss = 0.0

    lr_list.append(optimizer.param_groups[0]['lr'])

print("Finished Training")
```

In [ ]:
```
plt.figure(figsize=(12, 6))
plt.plot(list(range(len(lr_list))), lr_list, label="lr")
plt.xlabel("Epoch", fontsize=14)
plt.ylabel("Learning rate", fontsize=14)
plt.show()
```

3. Tuning siêu tham số

Cài đặt thư viện ray

In [11]:
```
import os
from torch.utils.data import random_split
from functools import partial
```

Viết hàm huấn luyện mô hình nơ-ron trên tập CIFAR10

In [15]:
```
def train_cifar(config, checkpoint_dir=None, data_dir=None):
    net = Net(config["l1"], config["l2"])

    device = "cpu"
    if torch.cuda.is_available():
        device = "cuda:0"
        if torch.cuda.device_count() > 1:
            net = nn.DataParallel(net)

    net.to(device)

    criterion = nn.CrossEntropyLoss()
    optimizer = optim.SGD(
        net.parameters(),
        lr=config["lr"],
        momentum=0.9
    )

    # Load checkpoint nếu có
    if checkpoint_dir:
        checkpoint = torch.load(
            os.path.join(checkpoint_dir, "checkpoint.pth")
        )

        net.load_state_dict(checkpoint["model_state_dict"])
        optimizer.load_state_dict(checkpoint["optimizer_state_dict"])
```

```
    trainset, testset = load_data(data_dir)

    test_abs = int(len(trainset) * 0.8)

    train_subset, val_subset = random_split(
        trainset,
        [test_abs, len(trainset) - test_abs]
    )

    trainloader = torch.utils.data.DataLoader(
        train_subset,
        batch_size=int(config["batch_size"]),
        shuffle=True,
        num_workers=8
    )

    valloader = torch.utils.data.DataLoader(
        val_subset,
        batch_size=int(config["batch_size"]),
        shuffle=True,
        num_workers=8
    )

    # Training
    for epoch in range(5):

        net.train()

        running_loss = 0.0
        epoch_steps = 0

        for i, data in enumerate(trainloader, 0):

            inputs, labels = data
            inputs, labels = inputs.to(device), labels.to(device)

            optimizer.zero_grad()

            outputs = net(inputs)

            loss = criterion(outputs, labels)

            loss.backward()

            optimizer.step()

            running_loss += loss.item()
            epoch_steps += 1

            if i % 2000 == 1999:
                print(
                    "[%d, %5d] loss: %.3f"
                    % (
                        epoch + 1,
                        i + 1,
```

```
                    running_loss / epoch_steps
                )
            )

            running_loss = 0.0

    # Validation
    net.eval()

    val_loss = 0.0
    val_steps = 0
    total = 0
    correct = 0

    with torch.no_grad():

        for data in valloader:

            inputs, labels = data

            inputs = inputs.to(device)
            labels = labels.to(device)

            outputs = net(inputs)

            _, predicted = torch.max(outputs.data, 1)

            total += labels.size(0)

            correct += (predicted == labels).sum().item()

            loss = criterion(outputs, labels)

            val_loss += loss.item()

            val_steps += 1

    accuracy = correct / total
    avg_val_loss = val_loss / val_steps

    print(
        f"Epoch {epoch+1}: "
        f"val_loss={avg_val_loss:.4f}, "
        f"accuracy={accuracy:.4f}"
    )

    # Save checkpoint
    save_dir = "./checkpoints"

    os.makedirs(save_dir, exist_ok=True)

    save_path = os.path.join(
        save_dir,
        f"checkpoint_epoch_{epoch+1}.pth"
    )
```

```
        torch.save(
            {
                "model_state_dict": net.state_dict(),
                "optimizer_state_dict": optimizer.state_dict(),
            },
            save_path
        )

        print("Saved checkpoint:", save_path)

    print("Finished Training")

    return {
        "loss": avg_val_loss,
        "accuracy": accuracy
    }
```

Lựa chọn siêu tham số tốt (hyperparameter tuning)

In [ ]:

```
import os
import itertools

def main(num_samples=10, gpus_per_trial=1):
    data_dir = os.path.abspath("./data")

    config = {
        "l1": [32, 64],
        "l2": [16, 32],
        "lr": [1e-4, 1e-2],
        "batch_size": [16]
    }

    keys = config.keys()
    values = config.values()

    all_configs = [
        dict(zip(keys, v))
        for v in itertools.product(*values)
    ]

    results = []

    for i, cfg in enumerate(all_configs):
        print(f"\n===== Trial {i+1}/{len(all_configs)} =====")
        print(f"Config: {cfg}")

        # gọi train
        result = train_cifar(
            config=cfg,
            data_dir=data_dir
        )

        results.append({
            "config": cfg,
            "result": result
```

```
        })

    best_result = max(
        results,
        key=lambda x: x["result"]["accuracy"]
    )

    print("\n===== Best Result =====")
    print("Config:", best_result["config"])
    print("Accuracy:", best_result["result"]["accuracy"])


main(num_samples=10, gpus_per_trial=1)
```

Cài đặt thử viện skorch

In [ ]:
```
!pip install skorch
import numpy as np
from sklearn.datasets import make_classification
from torch import nn

from skorch import NeuralNetClassifier
from sklearn.model_selection import GridSearchCV
```

Sử dụng thư viện skorch để huấn luyện

In [ ]:
```
trainset, testset = load_data('./data')
(X, y) = np.asarray(trainset.data[:]), np.asarray(trainset.targets[:])
X = X.reshape((-1, 3, 32, 32))
X = X.astype(np.float32)
y = y.astype(np.int64)

net = NeuralNetClassifier(
    Net,
    max_epochs=5,
    lr=0.01,
    # Shuffle training data on each epoch
    iterator_train__shuffle=True,
)

# training with default config
net.fit(X, y)
y_proba = net.predict_proba(X)
```

Dùng skorch để lựa chọn siêu tham số

In [ ]:
```
# hyperparameter tuning
net.set_params(train_split=False, verbose=0)
params = {
    'lr': [1e-4, 1e-2],
    'module__l1': [32, 64],
    'module__l2': [16, 32],
}
```

```
gs = GridSearchCV(net, params, refit=False, cv=3, scoring='accuracy', verbose=2)

gs.fit(X, y)
print("best score: {:.3f}, best params: {}".format(gs.best_score_, gs.best_params_))
```